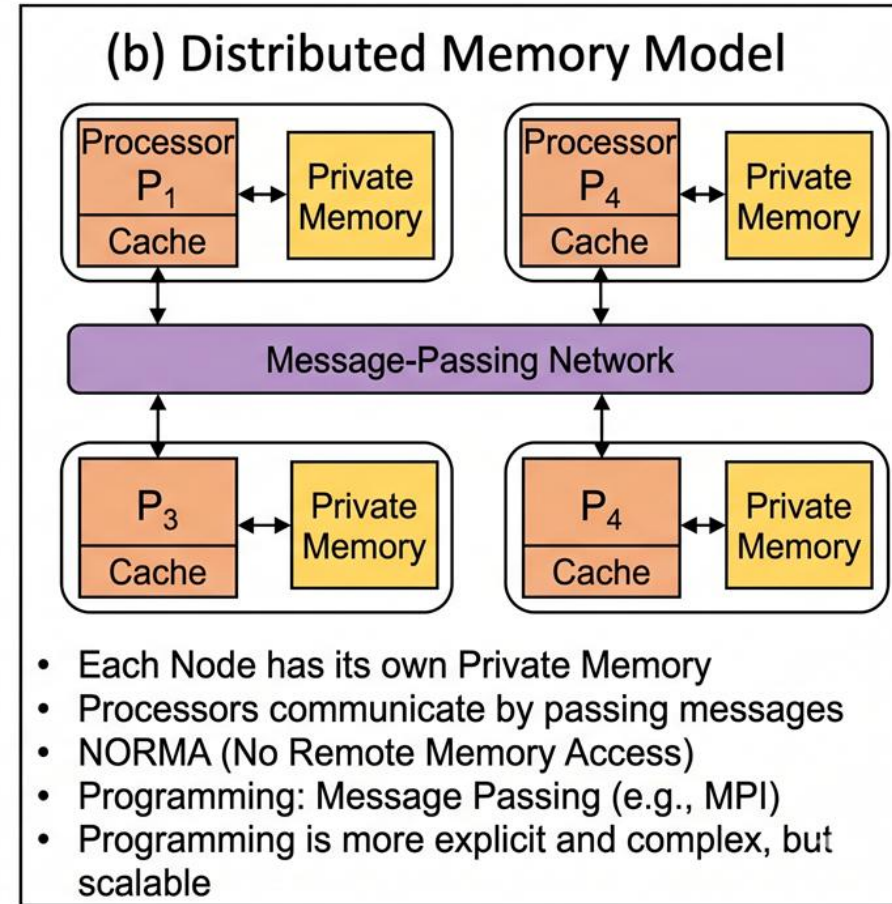
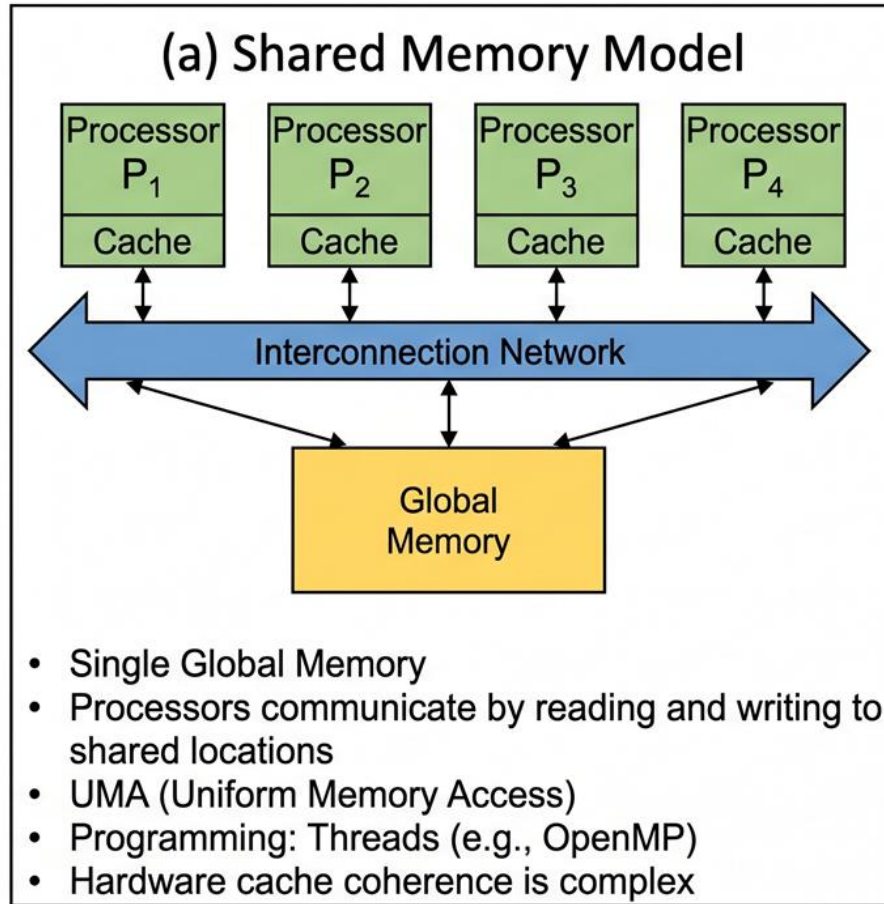


Message Passing Interface (MPI)

Agus Ambarwari

Shared and Distributed Memory in Parallel Computing



MPI (Message Passing Interface)

A programming standard or rule specifically designed to work on the **Distributed Memory Model**.

Facilitates how separate computers can coordinate with each other, exchange data, and solve large tasks simultaneously as if they were a single entity.

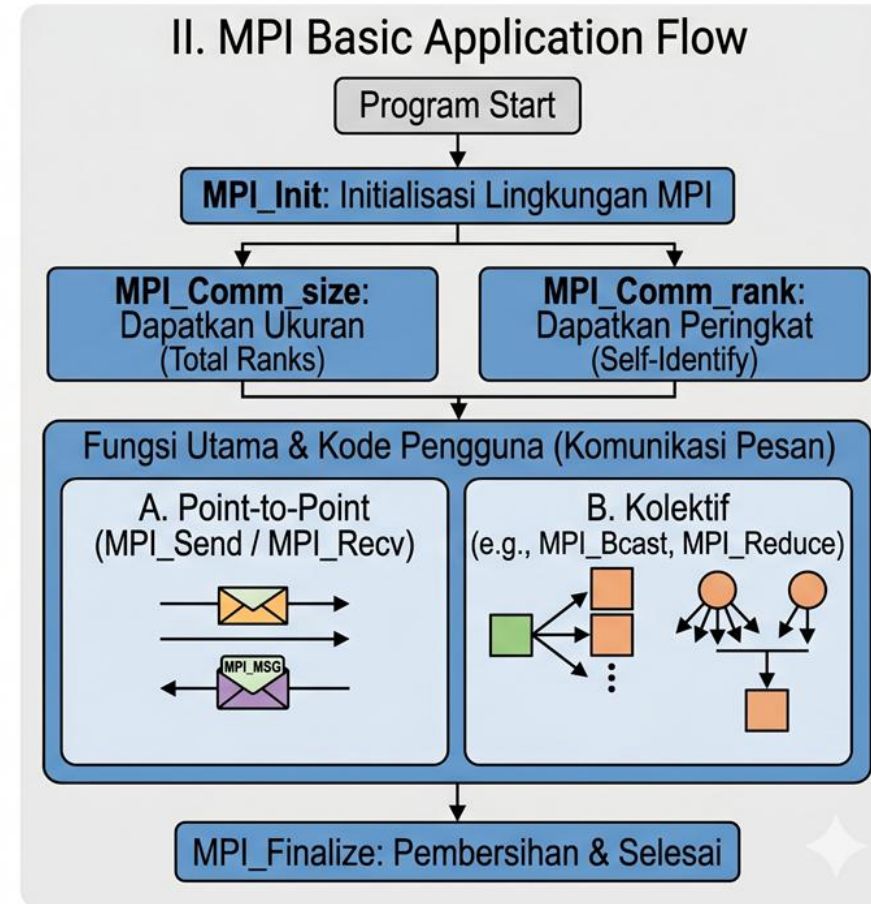
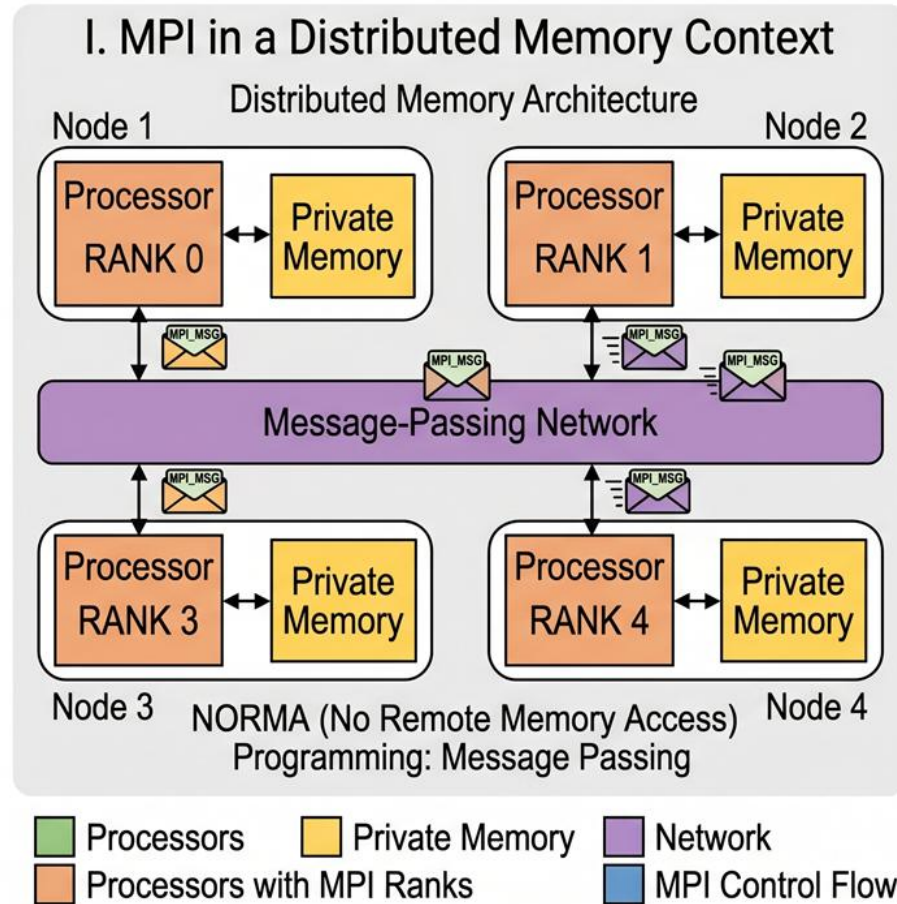
| The SPMD Execution Model

Single Program, Multiple Data

-  **One Code, Many Instances:** Write a single source code that is replicated and executed simultaneously by multiple processors.
-  **Self-Identification:** Since the code is identical, every process must determine "Who am I?" and "How many colleagues do I have?"
-  **Rank & Size:** Initialization sets up the **Rank** (unique ID) and **Size** (total process count) for each participating processor.

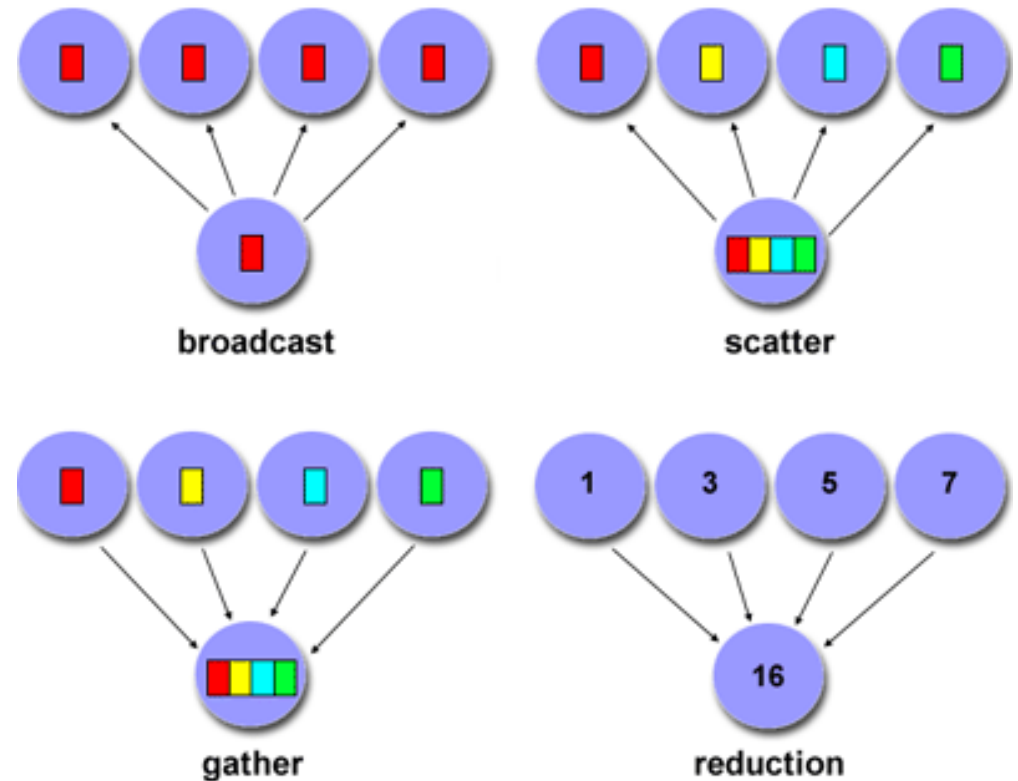


MPI: Context and Application Flow



MPI: Core Communication Type

- **Point-to-Point Communication:** Involves one sender and one receiver (e.g., Send/Receive).
- **Collective Communication:** Involves a group of processes, such as broadcasting data from one node to all others or collecting results.
 - **Broadcast (MPI_Bcast):** Sends data from a root process to all others.
 - **Scatter (MPI_Scatter):** Divides data on one process and distributes it.
 - **Gather (MPI_Gather):** Collects data from all processes to a root.
 - **Reduce (MPI_Reduce):** Performs global operations (e.g., sum, max) across nodes.



MPI Point-to-Point Communication

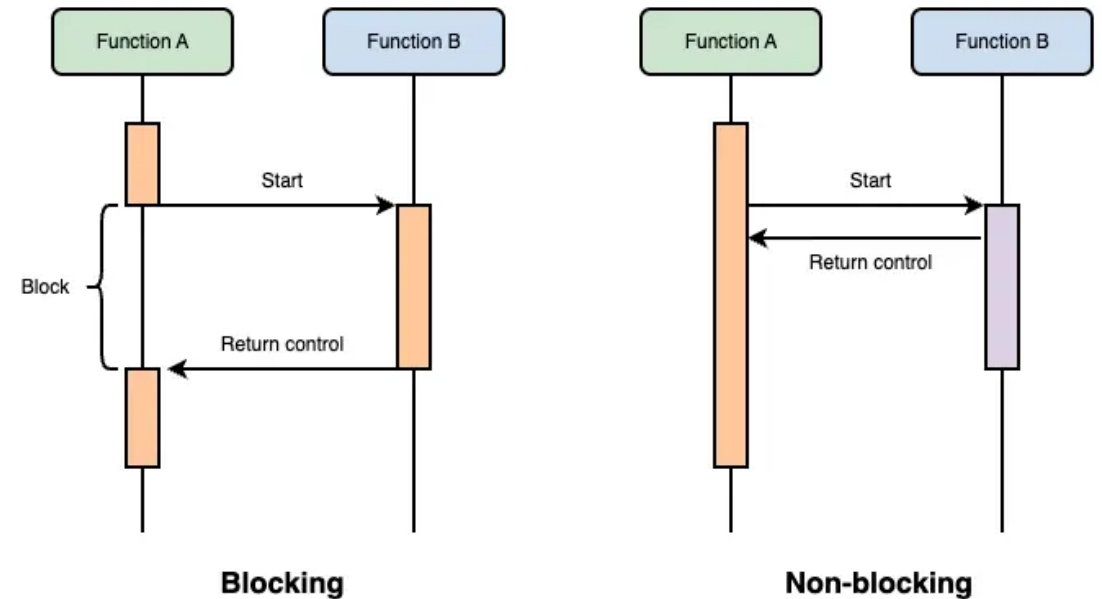
Blocking (MPI_Send/Recv):

When a function is called, the caller loses control and waits until the task is completed. During this time, no other tasks can be performed. **Synchronous**

Non-Blocking (MPI_Isend/Irecv):

The caller immediately regains control after the function call and can perform other tasks. The caller does not wait.

Asynchronous



P2P Blocking: Example Code

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, dokumen;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        dokumen = 100;

        // Proses diblokir hingga dokumen berpindah tangan ke Proses 1
        // Parameter: (data, jumlah data, tipe, tujuan, label pesan, jaringan)
        MPI_Send(&dokumen, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

        printf("Proses 0: Dokumen bernilai %d berhasil dikirim.\n", dokumen);
    }
    else if (rank == 1) {
        MPI_Status status;

        // Proses masuk wait state menunggu dokumen dari Proses 0
        // Parameter: (tempat_simpan, jumlah, tipe, sumber, label, jaringan, status)
        MPI_Recv(&dokumen, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

        printf("Proses 1: Dokumen bernilai %d diterima dan siap diproses.\n", dokum);
    }

    MPI_Finalize();
    return 0;
}
```

```
PS D:\PROJECTS\CPP_Parallel\MPI> mpiexec -n 2 .\p2p_blocking
Proses 1: Dokumen bernilai 100 diterima dan siap diproses.
Proses 0: Dokumen bernilai 100 berhasil dikirim.
```

MPI_Send	(buf, count, type, dest, tag, comm)	Menunggu hingga buffer aman digunakan kembali.
MPI_Recv	(buf, count, type, src, tag, comm, status)	Berhenti hingga pesan benar-benar diterima.

P2P Non-Blocking: Example Code

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>

int main(int argc, char** argv) {
    int rank, paket;
    MPI_Request request;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        paket = 500;

        // Titip paket ke kurir jaringan, langsung lanjut ke baris bawah
        MPI_Isend(&paket, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

        printf("Proses 0: Melakukan komputasi lain (background) mengatur matriks...\n");
        sleep(1); // Simulasi kerja berat selama 1 detik

        // Cek resi. Memaksa program menunggu sampai transfer benar-benar selesai
        MPI_Wait(&request, &status);
    }
    else if (rank == 1) {
        // Beritahu sistem untuk siap menerima paket, tapi lanjut kerja dulu
        MPI_Irecv(&paket, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &request);

        printf("Proses 1: Mengeksekusi instruksi independen sambil menunggu data...\n");
        sleep(2); // Simulasi kerja berat selama 2 detik

        // Cek resi penerimaan. Berhenti bekerja sampai paket benar-benar tiba
        MPI_Wait(&request, &status);

        printf("Proses 1: Buffer sinkron. Paket bernilai %d siap digunakan.\n", paket);
    }

    MPI_Finalize();
    return 0;
}
```

```
PS D:\PROJECTS\CPP_Parallel\MPI> mpiexec -n 2 .\p2p_nonblocking
Proses 0: Melakukan komputasi lain (background) mengatur matriks...
Proses 1: Mengeksekusi instruksi independen sambil menunggu data...
Proses 1: Buffer sinkron. Paket bernilai 500 siap digunakan.
```

MPI_Isend	(buf, count, type, dest, tag, comm, request)	Langsung kembali; butuh pengecekan status nantinya.
MPI_Irecv	(buf, count, type, src, tag, comm, request)	Menginisiasi penerimaan tanpa menunggu data tiba.
MPI_Wait	(request, status)	Memaksa proses menunggu hingga operasi asinkron selesai.

Continue...

- <https://github.com/ambarwariagus/Modul-Praktikum-CPP/blob/main/Modul%20MPI%201.md>
 - Scatter
 - Gather
 - Reduction